

The cost of certainty

Autonomous, general-purpose formal verification for the mechanisms that run markets and institutions, and the library it produces.

Daniel Alami

July 2026

Abstract

A machine-checked proof used to cost a specialist six months, and the specialists were mathematicians with every reason to formalize perfectoid spaces and none to formalize a credit-agreement covenant. So the covenant went unformalized, along with the waterfall, the quorum bound, the pricing invariant, and most of the logic that runs money and institutions. The people who could verify and the systems worth verifying were rarely the same.

This paper argues that a closed-loop, general-purpose verification apparatus collapses that cost toward the cost of compute, and it shows what falls out. A human writes a natural-language blueprint. From there the apparatus runs without a verification engineer in the loop: it formalizes the blueprint in Lean 4 through a faithfulness firewall, searches for a proof, and re-verifies every closure with a governance kernel the proving agent cannot influence. Autonomy starts at the blueprint, where the residual human judgment lives. The output is `nonmathlib4`: a growing library of kernel-checked theorems in finance, strategy, distributed systems, cryptography, compliance, social choice, and corporate governance, each self-contained, each reporting only the three standard axioms, none carrying a `sorry`. We table nineteen of them and walk through one in full: no round-trip arbitrage is possible at any state a constant-product market maker can reach.

The kernel certifies that the proof follows from the stated specification. It says nothing about whether the specification denotes the concept a human had in mind. Our Shamir secret-sharing result sits on that line: it proves the algebraic secrecy bijection, not the measure-theoretic Shannon statement. Owning that boundary is what separates a verified claim from an oversold one.

1. The verification gap

You trust most of the systems you rely on for the same reason: someone checked, and you take their word. The tests passed. The auditor signed off. The lawyers initialed every page of the credit agreement. Call it trust by vouching, and it carries nearly all software, contracts, and financial machinery. It is also how they fail quietly, when the check missed a case no one thought to try.

A second kind of assurance is stronger, and you have almost never seen it applied to a covenant or a settlement rule. A proof assistant like Lean 4 (de Moura and Ullrich) checks every step of an argument against a small trusted kernel; a statement it accepts is as certain as anything in engineering gets, and it leaves a proof a skeptic can re-run. The reason it stayed inside mathematics and safety-critical hardware is cost. Producing the proof is hard, and the people who can do it are few.

The community that built Mathlib, Lean's library of mathematics (mathlib Community), is mathematicians; they formalized the definition of a scheme and the theory of perfectoid spaces (Buzzard, Commelin, and Massot). The lawyers and bankers who work with waterfalls and covenants do not write Lean. On one side sit the people who can verify. On the other sit the systems worth verifying. They rarely meet. The seL4 microkernel, a few thousand lines of operating-system code, took a team on the order of twenty person-years to prove correct (Klein and colleagues), which is why almost nothing outside safety-critical systems ever got the treatment.

That price is what changes now. When the marginal cost of a proof falls from months of a specialist's time to a few dollars of model API calls, the calculus flips. Verifying a covenant stops being a capital project and becomes something you run the way you run a linter on a pull request. The question is no longer "can we afford to verify this?" but "why wouldn't we?"

Two conditions make the fall real, and both have to hold. The loop has to run on its own once a human writes the blueprint, with no formal-methods engineer steering the formalization, the search, or the check. And the prover has to be general, one system that proves a market-maker invariant, a quorum bound, and a tax rule rather than a bespoke tool for each. An autonomous prover locked to one domain solves a narrow problem; a general prover that still needs an expert to drive it has not moved the cost. Only both together do.

Michael Porter drew a distinction that fits. Operational effectiveness is doing the same activities better than rivals; strategic positioning is doing different activities, or the same ones differently, in a way rivals cannot easily copy. A faster prover is operational effectiveness, and it erodes with the next model release. The position that holds is the machinery that makes an untrusted model's output trustworthy — a different activity, and one that gets better as the models do instead of eroding.

2. The unoccupied combination

Formal verification of business, financial, and legal logic is not new. Several of these systems run in production, and one built a business.

Imandra, from Aesthetic Integration, is a commercial theorem prover for financial algorithms. It reasons about trading systems and venue logic for regulatory compliance under regimes like MiFID II, and it shipped the first formal verification platform aimed at smart contracts. AWS Zelkova encodes the semantics of IAM and S3 access policies into SMT and answers questions like "can any principal outside this account read this bucket?" It runs millions of times a day, and its companion Tiros reasons about network reachability. Catala, from Inria, is a language for writing tax and benefits law as executable, verifiable code, and the French tax administration uses it. For smart contracts, the Certora Prover checks Solidity and EVM bytecode against specifications written in its own CVL language and runs inside production audit workflows, while KEVM, from Runtime Verification, gives the EVM a complete formal semantics in the K framework. And the systems-verification tradition, seL4's machine-checked microkernel and the DeepSpec line of verified software, is the proof that whole real artifacts can be certified end to end.

So the frontier is occupied. What is not occupied is the combination this apparatus targets. Every system above is either domain-specific, meaning it verifies one class of thing with a tool built for that class, or expert-driven, meaning a trained verification engineer writes the specification

and steers the proof. Zelkova verifies access policies and nothing else. Certora needs a security engineer to write CVL. Catala needs someone fluent in Catala. The apparatus here is autonomous after the blueprint, no verification engineer in the loop, and general-purpose, plain Lean 4 with Mathlib rather than a domain solver. That pairing lets one system produce an AMM theorem, a bankruptcy waterfall, a Byzantine bound, and a secret-sharing guarantee from natural-language blueprints alone.

Table 1 places the work against them.

Table 1. Where this work sits.

System	Prover	Who drives it	What it verifies
Imandra	Domain solver (OCaml-based)	Domain user	Trading-algorithm and venue logic
AWS Zelkova / Tiros	SMT (domain-encoded)	Automated, single class	Access-policy and reachability queries
Certora Prover	SMT over CVL specs	Security engineer writes CVL	EVM smart-contract properties
KEVM	K framework	Verification expert	EVM semantics and bytecode
Catala	DSL for law	Lawyer-programmer	Tax and benefits rules
seL4 / DeepSpec	Isabelle / Coq, hand-driven	Verification team	OS kernel, verified software stack
LeanDojo, AlphaProof, DSP	Lean / Isabelle	Automated, trusted benchmarks	Competition mathematics (statement given true)
EconCSLib	Lean 4 + Mathlib	AI-assisted, human-curated	Reusable econ definitions and theorems (game theory, mechanism design)
This work	Lean 4 + Mathlib	Autonomous after the blueprint	Dynamic mechanisms, governance-re-verified, faithfulness boundary disclosed

Every prior system is domain-specific, or expert-driven, or restricted to trusted benchmarks where the statement is known true, or confined to static definitions. The combination in the last row, a general-purpose prover plus post-blueprint autonomy plus a governance loop that re-checks every closure plus explicit handling of the faithfulness boundary, is the position none of the others occupy.

The apparatus also draws on a second literature, on autonomous proving and autoformalization. Autoformalization, translating natural-language mathematics into a formal system, was shown practical with large language models by Wu, Jiang, Li, Rabe, Staats, Jamnik, and Szegedy, who translated a quarter of competition problems perfectly into Isabelle. Draft, Sketch, and Prove, from Jiang and colleagues, used an informal proof to guide a formal prover and roughly doubled the success rate on the miniF2F benchmark. LeanDojo, from Yang and colleagues, built the retrieval-augmented proving stack for Lean that later work stands on. Thor and Magnushammer, from Jiang, Mikula, and their coauthors, wired language models to premise-selection hammers. HyperTree Proof Search, from Lample and colleagues, brought AlphaZero-style search and expert iteration to theorem proving, and DeepMind's AlphaProof carried that line to a silver-medal score at the 2024 International Mathematical Olympiad, training on eighty million autoformalized statements.

Prior autoformalization and autonomous-proving work optimizes closure rate on trusted benchmarks, where a competition problem is known to be true and a compiling proof is the whole game.

This apparatus operates in the untrusted regime, where a compiling proof is necessary but not sufficient, because the statement was written by the same fallible model that wrote the proof. There a proof search alone is not enough: the Lean compiler grounds the model adversarially, forcing it to discard hallucinations; a governance kernel re-checks every closure; and a firewall gates every statement before a proof is ever attempted. The model proposes and the kernel disposes.

Two 2026 systems share parts of the approach. LeanMarathon formalizes research mathematics autonomously and audits its own statements: a reviewer agent checks each Lean type against the source paper, and a build gate keeps the natural-language and formal dependency graphs from drifting apart. It works from rigorous papers, where the target statement is already precise, so the task is to verify a faithful translation, not to decide what the statement should be. Coelho's mathematical-finance library carries the same two trust signals reported here, faithfulness classification and an axiom-pinning gate, but it is hand-written and stays in the pricing and stochastic-calculus core. The regime this apparatus is built for is the one neither enters: a non-mathematical intent, stated in prose by someone who does not write Lean, where the formal statement can be a plausible-but-wrong rendering of a covenant or a policy and no rigorous source exists to check it against. When the model writes both the statement and the proof, a clean proof of the wrong statement is the failure mode, and the load-bearing layer is the governance kernel that re-checks every closure against a statement the model itself produced.

This is a library for the mechanisms markets and institutions run on, the dynamic multi-step objects rather than a grab-bag of financial lemmas. An automated market maker is a state machine you trade against. A waterfall is a distribution procedure. A quorum system is a fault-tolerance protocol. These are mechanisms with reachable states and adversarial histories, not static facts.

That framing also fixes the relationship to the one adjacent library. EconCSLib, from Bei, Ma, Jing, Fu, and Tang, is an early, AI-assisted Lean 4 library for computational economics. EconCSLib supplies reusable interfaces and definitions for players, preferences, utilities, allocations, and mechanisms, the vocabulary a domain needs before it can state a theorem. This work proves the dynamic, multi-step mechanisms end to end and autonomously: the AMM that survives any trade history, the waterfall that stays feasible, the quorum that intersects. The two are complements, and a mature version of the field wants the shared vocabulary and the proved mechanisms both, with their interfaces meeting.

3. The apparatus

We built the apparatus in four stages: formalize, solve, govern, self-learn. One design rule runs through all four: put determinism exactly where soundness requires it, at the trust boundary, and give the agent full freedom everywhere upstream. Too much determinism and you hand-wire the prover into uselessness. Too little and the agent becomes its own judge, which is how a wrong proof gets waved through. Knowing which knob is which is the trick, and the whole architecture is our answer to it. Throughout, "we" refers to the authors who built and configured the apparatus; "the apparatus" refers to the running system, which operates without human intervention once the blueprint is written.

Formalize. The input is a blueprint, a natural-language description of the theorem and the vocabulary it needs. The apparatus renders it into a Lean statement. Then, before any proof is attempted, a faithfulness firewall decides whether that statement means what the blueprint said.

The firewall runs a stack of checks. The statement has to compile. It has to be non-trivial, so a cheap tactic cannot close it and it is not vacuously true. Its structure has to match the reference, meaning the binder counts, the conclusion's operator, and the quantifier order line up, which is how a dropped hypothesis or a weakened conclusion gets caught. For decidable policy fragments the firewall goes further and returns a typed certificate: either the two formulations are provably equivalent over the whole domain, or here is a concrete input that tells them apart, or the question sits outside the decidable fragment and is declared as such rather than silently waved through. Last, the statement is back-translated to natural language and a separate judge has to rule it the same problem. Only then does a proof get attempted.

Most of these checks are deterministic: the compile check, the non-triviality check, the structural comparison, and the decidable-fragment decision procedure run in Lean, a small amount of Python, and for policy fragments a z3 or Gröbner routine, none of which the proving model can influence. Only the back-translation check is itself a language model, and it is drawn from a different model family than the one that wrote the statement, so a single model cannot both introduce an error and clear it. The governance stage described below uses no model at all.

The firewall guarantees fidelity, not truth. Every check answers one question: does the Lean statement mean the same thing as the blueprint? None of them answers the prior question of whether the blueprint itself is a correct statement of the intended result. That question is undecidable in general and belongs to the human.

Solve. A proof search runs over the formal statement. The engine is agentic: a frontier model, equipped with tools and a warm Lean session, decides at each step whether to prove the goal directly, decompose it into sub-lemmas, or, if it judges the goal false, prove its negation. When it decomposes, the kernel audits the decomposition, checking that it is **sorry**-free, non-circular, uses every lemma, and actually proves the parent, before the sub-lemmas are attacked in turn. A pool of diverse proposers can attack a goal in parallel, but only one serial verifier commits, first valid proof wins. The search has the usual apparatus, best-first frontier scoring, restarts, value backup, but none of it touches soundness. It only decides what to try next.

Govern. This is the boundary, and it is the part a stronger model cannot give you, because the model is the thing being policed. A **closed** verdict from the solver is a proposal, nothing more. It becomes a result only after a fixed stack of deterministic checks the agent cannot influence:

- The Lean kernel re-verifies the proof term.
- `#print axioms` on the closed theorem must report a subset of `{propext, Classical.choice, Quot.sound}`. A **sorry** leaves **sorryAx** in that list and is rejected. **native_decide** leaves **Lean.ofReduceBool** and is rejected. A smuggled custom axiom is rejected.
- The proved statement, and every definition it rests on, must be unaltered: no definition swapped for a shell, no notation or instance that quietly changes what the symbols mean.
- The proof must need the blueprint's own vocabulary. A proof that compiles against bare Mathlib was a library lookup, not a closure, and the matched-negative-control catches it.
- The statement must not be vacuously true.

An adversarial red-team confirms the boundary holds. Every attempt to smuggle an unsound proof past it, a bare **sorry**, a nested **sorry**, a **native_decide** reduction, a false custom axiom cited as if proven, was rejected, and the clean control passed: five out of five caught, zero false flags. On an

open target where the model produced three proofs that compiled with clean axioms and no `sorry`, the statement-integrity check rejected all three, because each had faithlessly altered the statement it claimed to prove. A verifier that trusted "it compiled" would have accepted all three.

Self-learn. Every kernel-closed theorem is banked into a library the next proof can cite, and this is how the library compounds. The effect is a change in complexity, not only in convenience: banking a lemma converts a combinatorial re-search into a linear composition. A later result does not re-derive the constant-product bound; it cites the banked lemma and proves only the new content on top. This is the mechanism behind the numbers in Table 2, where the constant-product invariant took about five hours to establish and the no-round-trip-arbitrage result that stands on it closed in under nine minutes. The library is the environment the engine proves against, and it grows.

One property of the boundary makes agent-generated proofs safe to trust. Since soundness lives entirely at the governance stage, everything upstream is free. A false sub-lemma the agent invents simply fails to close. A bad decomposition dead-ends. The worst an aggressive, hallucinating agent can do is leave a recorded gap, never a false theorem. So we can let a frontier model run wild in the search and still stand behind every closure.

4. The evidence

`nonmathlib4` is the library the apparatus produced. Each file imports `Mathlib`, stands alone, and carries a generated provenance header recording the outcome, the axioms, the wall-clock time, the phase breakdown, and any banked lemmas it reused. The headers are emitted from run telemetry, not written by hand. Table 2 lists nineteen filed theorems across eight domains.

Table 2. Filed results in `nonmathlib4`. All report `#print axioms` $\subseteq$ `{propext, Classical.choice, Quot.sound}` and carry no `sorry`. Wall time is launch-to-close for the filed run; a campaign family may span several runs.

Domain	Theorem	What it means	Wall (filed run)
Finance	<code>constantProductAMM_temporal_invariant_and_no_roundTrip_profit</code>	For an $x \cdot y = k$ pool with fee $\gamma \in (0, 1]$, the product never decreases across any adversarial finite trade sequence, and a single round-trip returns at most the input, strictly less with a real fee.	~18,457 s (~5.1 h)
Finance	<code>no_history_enables_round_trip_arbitrage</code>	No trade history reaches a state where a round-trip becomes profitable. Cites the banked constant-product rungs rather than re-deriving them.	~486 s
Finance	<code>cost_eq_statePriceWeighted_payoff</code> (FTAP, easy direction)	A strictly positive state-price vector pricing every asset implies no arbitrage.	~336 s
Finance	<code>statePriceVector_exists_of_noArbitrage</code> (FTAP, hard direction)	The converse: absence of arbitrage forces a strictly positive state-price vector to exist (Stiemke/separation argument, constructed from scratch — <code>Mathlib</code> has no finite Farkas). Completes the FTAP as a machine-proved iff.	~2,825 s (family ~4,814 s over 2 runs)
Finance	<code>waterfallDistribution_feasible_of_linearOrder</code>	Under a strict ranking of claims, the absolute-priority waterfall is feasible: senior claims paid before any junior claim receives anything.	~4,413 s

Domain	Theorem	What it means	Wall (filed run)
Finance	<code>pariPassuWaterfallDistribution_feasible_and_ranked_absolutePriority</code>	Extends the waterfall to equal-ranking (pari-passu) tranches with pro-rata sharing within a tranche.	~1,412 s
Strategy	<code>topkisObjective_parametricArgmaxSet_isSublatticeSet</code>	Topkis' theorem: for a supermodular objective with increasing differences, the parametric argmax set is a sublattice.	~1,124 s
Strategy	<code>ordinalTopkis_compact_existence_and_strongSetMonotone_explicit</code>	The ordinal (single-crossing) generalization: optimal choices are monotone in the strong set order.	~647 s
Distributed systems	<code>byzantine_threshold_quorum_safe_available_iff_and_tight_witness</code>	A quorum threshold that is both safe and available exists iff $n \geq 3f+1$, with a tightness witness.	~804 s
Cryptography	<code>shamir_threshold_reconstruction_secretcy_tightness</code>	Shamir (t, n) sharing: reconstruction from any t shares, a perfect-secrecy bijection from any $t-1$, and boundary tightness.	~1,172 s
Compliance	<code>restricted_payments_permission_safety_invariants</code>	A real SEC EDGAR restricted-payments covenant: the accounting abstracted from natural language, its safety invariants proved (a default blocks every payment; an exhausted basket blocks any further one).	~509 s
Social choice	<code>median_voter_theorem</code>	Black's median voter theorem: with an odd electorate and single-peaked preferences over a linearly ordered policy space, the median voter's ideal point is a Condorcet winner — it beats every alternative in pairwise majority vote.	~465 s (family ~5,184 s over 2 runs)
Compliance	<code>leverage_backstop_subsumes_risk_weighted_floor</code>	Basel III leverage ratio as a non-risk-based backstop: whenever the effective risk weight is low enough that the risk-weighted floor does not dominate ($8 \cdot w \leq 3$, i.e. $w \leq 37.5\%$), any leverage-compliant bank automatically meets its risk-weighted capital requirement too — so gaming risk weights toward zero cannot escape the capital the flat 3% leverage floor independently demands. A second regulator (banking capital) alongside the securities covenant.	~295 s
Finance	<code>reachable_state_solveny_guarded_actions</code>	DeFi liquidation safety as a reachable-state guarantee: from any healthy collateralized-debt position, every state reached by an admissible sequence of guarded operations (deposit, borrow, repay, withdraw) stays healthy and non-liquidatable, and the per-step guard is load-bearing — dropping it admits a borrow into an unhealthy state. A dynamic solvency property over adversarial trajectories, not a static inequality.	~822 s (family)

Domain	Theorem	What it means	Wall (filed run)
Corporate governance	<code>corporate_governance_present_basis_two_gate_and_basis_divergence</code>	Charter adoption is a genuine two-gate conjunction (quorum and a present-basis supermajority), and the <i>basis</i> is load-bearing: a tally can clear the supermajority on votes actually <i>cast</i> yet fail it on shares <i>present</i> , so the same votes adopt under one reading of the charter and defeat under the other — the divergence driven entirely by a strictly positive abstention. A governance procedure, not a preference-aggregation theorem.	~1,305 s
Finance	<code>trace_scanl_uncrossed</code>	Central limit order book, no-crossing safety: a resting book that starts uncrossed (best bid below best ask) stays uncrossed at every state reachable by an arbitrary finite sequence of rest and cancel operations under price-time priority — the market-microstructure invariant that a matching engine never leaves a marketable order resting against the opposite side. A dynamic safety property over adversarial operation trajectories, <code>bestBid/bestAsk</code> defined as order-independent list extrema (<code>List.max?</code>) rather than queue heads.	~2,085 s (family: 34 runs, ~229 min active-solve, 37 rung-closures — capstone assembled from banked rungs, ratified directly)
Strategy	<code>vcg_dsic_pivot_independence_and_twoUnit_witness</code>	The VCG mechanism selects a welfare-maximizing allocation and charges each agent the Clarke pivot payment; the pairing is dominant-strategy truthful, and an agent's pivot benchmark is independent of that agent's own report (pivot independence). A concrete two-unit witness keeps the guarantee non-vacuous: a profile where misreporting changes the allocation, truthful reporting does not lower the agent's own utility, and the pivot payment charged is strictly nonzero.	~1,336 s
Probability	<code>base_rate_governs_positive_predictive_value</code>	For a screening test with sensitivity <code>se</code> , specificity <code>sp</code> , and prevalence π , the positive predictive value clears one-half exactly when the true-positive mass meets the false-positive mass ($se \cdot \pi \geq (1-sp) \cdot (1-\pi)$); and for any imperfect specificity, however small, there is a low enough prevalence at which even a perfectly sensitive test's positive result is more likely wrong than right — the base-rate fallacy, mechanized over an ordered field.	~2,706 s
Compliance	<code>le_permits_the_exact_boundary</code>	The faithfulness companion to the restricted-payments covenant: a laundered $\leq$ misreading of the builder-basket cap would permit a payment landing exactly at the basket ceiling, which the faithful covenant (a strict inequality) forbids — a concrete distinguishing transaction, not an assertion, showing the strict/non-strict distinction is load-bearing.	leaf-closed (outside the campaign loop)

The cost is worth stating plainly. The AMM temporal result is the expensive one: about 5.1 hours of wall time, 40 of 104 attempts closing, the rest recorded as failures. That is not a clean 40-out-of-40 run, and it should not read as one. The hardware is a single four-core AMD EPYC virtual machine with 15 GB of RAM and no GPU. Proof search dispatches frontier models through their APIs, OpenAI Codex-class models and Anthropic Claude Opus, as the agent; Lean elaboration and the kernel run on that machine. Wall time is dominated by API latency and Lean compilation rather than local compute, so a proof costs the API calls it makes, not a cluster reservation. The companion no-cyclic result closed in about 486 seconds, because it cited the banked rungs from the first proof and had only the new content to establish. The FTAP and ordinal-Topkis results closed in single-digit minutes. The first proof of a theory is expensive, and everything that compounds on it is cheap. Measured against the alternative of a specialist at six months, even the 5-hour case is a rounding error, and the marginal proof on an established theory costs minutes.

4.1 A worked walkthrough of the AMM no-arbitrage result

The AMM no-cyclic theorem is the most business-relevant result in the library, and it shows the apparatus and the compounding in a single object.

The setting. A constant-product automated market maker holds two reserves, x and y , and quotes trades that keep the product $x \cdot y$ at least constant. It charges a fee γ , a number in $(0, 1]$, where $\gamma=1$ is a feeless pool and $\gamma<1$ is a pool that keeps a real cut. Traders swap one token for the other. A round-trip is the natural attack: swap a token out, then immediately swap the proceeds back, hoping to end with more than you started. If that ever pays, the pool has an arbitrage, and someone will drain it.

The blueprint. The natural-language input asks for one thing. Over a pool with strictly positive reserves and a fee $\gamma \in (0, 1]$, prove that no trade history enables round-trip arbitrage. For every well-formed starting pool, every finite prior sequence of trades in any direction, a flash loan, a sandwich, a cyclic path, and every subsequent round-trip in either direction, the round-trip returns at most the input, and strictly less whenever the fee is real and the input is positive. Equivalently: at no reachable state can a trader close a loop against the pool for a strictly positive risk-free return. The blueprint spells out that this must cite the already-banked constant-product theory rather than re-derive it.

The statement. That blueprint becomes a Lean theorem, `no_history_enables_round_trip_arbitrage`, a conjunction of two parts. The first is universal: for every fee, every pool, every finite trade list, and every subsequent round-trip, the return is at most the input, and strictly less under a real fee with positive input. The second is an existence witness that keeps the claim non-vacuous: a concrete pool, fee, and trade where the strict inequality actually bites, so the theorem is not true merely because its hypotheses are never met. The firewall gates this statement, and it clears, because it means what the blueprint said.

What "reachable" buys you. The earlier result proved the round-trip loses at a *well-formed* pool. This one proves it loses at every pool the trader can actually face, which is the earlier pool after some arbitrary prior history. The bridge is a banked lemma, `executeTrades_keep_wellFormed`: executing any finite trade list preserves well-formed reserves throughout. So every reachable pool is well-formed, the banked round-trip bound applies to it verbatim, and the proof is a one-step composition. Instantiate the banked bound at the reachable pool, discharge its well-formedness

hypothesis with the banked invariant, cover both directions and the strict real-fee case. The new content is the quantification over histories. Everything else is cited.

Why it compounds. The library behaves as an environment rather than a pile. The no-cyclic proof reuses the round-trip- $\leq$ -input lemmas, the strict real-fee lemmas, and the sequence invariant, all kernel-clean rungs from the constant-product campaign. It re-derives none of them. So it closed in minutes where the foundation took hours, and the marginal cost of the next verified property keeps falling as the library grows.

The receipt. The closed theorem reports `#print axioms = {propxt, Classical.choice, Quot.sound}`, carries no `sorry`, and elaborates standalone against Mathlib. Anyone with the toolchain can run `lake env lean amm_no_cyclic_arbitrage.lean` and watch the kernel accept it. The guarantee is not "a strong model said so." It is a proof a small trusted kernel checked, re-runnable by a skeptic.

4.2 Modular constraints

Constraints are where a clean theorem meets a real system, and a library of only pristine baselines does not say which constraint breaks which result. The design answer is modularity: the baseline theorem and the constrained variant are separate, importable objects, so a practitioner can take the pure result and test which constraint their system actually faces breaks it. The pattern is already visible in the filed work. The pure constant-product no-arbitrage bound is one theorem; the "any adversarial history, any reachable state" version is another that imports it. The Byzantine bound is proved for a general adversary of size f and then instantiated at the tight $n=3f+1$ witness. The natural continuations follow the same shape: baseline VCG plus a budget cap, a clean settlement plus integer truncation, an honest-majority protocol facing Byzantine actors. The library should make it cheap to import the baseline and prove what survives the constraint, and costly to conflate the two.

5. The faithfulness boundary

A verified theorem says one thing: the proof follows from its specification. Whether that specification denotes the concept a human meant is a separate question, and the kernel never touches it. Everything the apparatus guarantees sits on the formal side of that line; everything a user cares about sits partly on the other side, in the blueprint, argued in prose. The Shamir result makes the line concrete.

Our Shamir secret-sharing theorem, `shamir_threshold_reconstruction_secretcy_tightness`, is kernel-clean and true. It proves three things over an arbitrary field: any t shares reconstruct the secret; for any $t-1$ observed shares there is exactly one degree- $<t$ polynomial consistent with each candidate secret, so secrets and consistent polynomials are in bijection; and at the $t-1$ boundary two distinct secrets each remain possible, so the bound is tight. The middle leg is named `PerfectSecrecy` in the file, and it states existence and uniqueness in full, an $\exists!$, not a cardinality proxy.

Now the caveat. `PerfectSecrecy` proves the *algebraic* secrecy bijection. It does not prove the *measure-theoretic* Shannon statement, that the posterior probability of the secret given $t-1$ shares equals its prior. The bijection is the capacity for perfect secrecy; the probabilistic conclusion

needs one more thing the development deliberately omits, uniform sampling of the polynomial's coefficients, which requires a finite field and a probability space and the measure theory that comes with it. The file uses `[Field F]`, correct for the bijection, insufficient for the sampling step. Separately, the reconstruction guarantee assumes the submitted shares are the dealer's own, so it models basic honest-but-curious Shamir rather than verifiable secret sharing, where a forged share is the whole threat.

Read carelessly, the symbol `PerfectSecrecy` over-claims. It reads like the full cryptographic property while it proves the algebraic core. A reader might call this laundering, a proof dressed up as more than it is. That reading misses the boundary. The gap traces entirely to a scoping choice in the blueprint. The blueprint asked for the algebraic bijection, so the apparatus proved the algebraic bijection, and the firewall correctly certified that the Lean statement means what the blueprint said. The kernel did its job. The apparatus did its job. The scope was set by the human who wrote the blueprint, and the file's own scope caveat says so in plain language.

The system enforces one discipline around this. The blueprint is the single highest-leverage and highest-risk input in the pipeline. A faithful formalization of a mis-scoped or over-named blueprint yields a true theorem that a reader can over-read. No governance check catches it, because nothing is formally wrong to catch: the proof is valid, the statement is faithful to the blueprint, the axioms are clean. The error lives in the denotation, in whether `PerfectSecrecy` should mean the algebraic bijection or the Shannon statement, and denotation is argued rather than proved. The apparatus surfaces this frontier rather than hiding it. It runs a denotation pass that reports whether a built definition is pinned by a verified external anchor or merely self-consistent, and when a definition is only self-consistent it says so, as a flagged gap, never as a certification. The filed Byzantine result carries exactly such a flag on one auxiliary definition, disclosed in its README.

A simple rule falls out. Trust the kernel for what it certifies, that the proof follows from the specification. Read the blueprint and the scope caveat for what the specification means. A verified theorem is a conditional promise: *if* the specification denotes what you think it does, *then* the property holds, mechanically and beyond doubt. The apparatus makes the "then" free. It cannot make the "if" free.

6. Applications

The apparatus points at several domains. We present four as illustrative options and one as the frontier, and we rank them by how well the current machinery fits, because pretending they are equally ready would repeat the over-claim the faithfulness boundary warns against.

Operating-covenant verification. A credit agreement carries covenants: leverage ratios, coverage tests, restricted-payment baskets. Whether a proposed transaction breaches a covenant is a question with a definite answer, and the waterfall results in the library are a first step into exactly this capital-structure logic. Formalizing the covenant faithfully takes the effort; the proof that follows is short.

IAM and access-policy state transitions. "Can a terminated contractor read production after this role change?" is a decidable question over linear and boolean constraints, which is where SMT quantifier elimination is strongest. This is the most decidable of the four, and it is also the most occupied. AWS Zelkova already does access-policy verification at production scale. The opening,

if there is one, is in multi-step transition logic and in being general-purpose, not in out-competing a tool that runs millions of times a day on the static case.

Supply-chain and ledger integrity. Conservation invariants, no double-spend, reconciliation across a distributed ledger, these are the same shape as the AMM invariants, quantities that must be preserved across an adversarial sequence of operations.

Option-value and pricing invariants. No-arbitrage bounds, monotonicity of value in the underlying, and put-call parity are theorems with clean statements the prover can already reach, starting from the filed FTAP result.

A contested-spec covenant, demonstrated. The most telling case is not the most decidable one but the one where the proof is easy and the faithfulness is hard. We ran one: a restricted-payments covenant from a live SEC EDGAR credit agreement. From a natural-language instruction the engine abstracted the accounting into opaque givens, proved the covenant's two safety invariants (a continuing default blocks every payment; an exhausted builder-basket blocks any further one), and a companion faithfulness certificate refutes each plausible misreading — $\wedge \rightarrow \vee$, a dropped basket cap, $\leftarrow \rightarrow \leq$ — with a concrete transaction it would wave through. Abstracting the accounting is a deliberate cut, not a dodge: the covenant failures that strip creditors are compositional — the basket-and-carve-out interlocks a liability-management loop exploits, not a mis-added balance — so proving the permission logic targets the failure mode that matters and keeps the state space searchable. Two limits remain: the abstraction level is the human's scoping choice, not the full financial model; and a thousand-page agreement at full fidelity is out of reach — that gap is the state of the art, per the limitations below.

7. Limitations

Autoformalizer at scale. Turning a hand-written blueprint into a faithful Lean statement works well for a single theorem. Turning an unstructured thousand-page regulation or a full production contract into faithful statements at that volume is the least-validated of the four stages, and it is what stands between the apparatus and a production deployment. The proofs themselves are past toy, the filed results are multi-lemma campaigns, but each was driven from a blueprint a human wrote. The distance the apparatus covers for one theorem it has not yet covered for a full document.

Denotation. The guarantee stops at the formal boundary. Whether a specification denotes the intended concept is argued, not proved, and no amount of proving capability changes that. It is a permanent property of what formal verification is, not a bug to be fixed, and the responsible move is to surface it rather than paper over it.

Clean versus messy specifications. The apparatus is strongest where the specification is clean: an AMM invariant, a quorum bound, a pricing theorem. It is weakest where the specification is contested, ambiguous, or buried in prose, which is exactly where the highest-value business logic lives. The contested-spec covenant we proposed as the next frontier expresses this limit directly. Some deterministic search moves are also redundant on easy targets, where a native tactic already closes the goal; the specialized levers earn their keep only where the plain cascade is blind.

The search/governance decoupling. The proof search delegates entirely to exogenous frontier models: Codex-class and Claude agents propose every move, and a shell-equipped model could reproduce those moves itself. The edge is not there, and not in any proprietary weights. It is in the

deterministic governance kernel and the faithfulness firewall, the layers that re-check every closure and gate every statement before a proof is attempted, which no model does for itself. A faster prover slots in behind them, and the guarantee holds regardless of which model is driving.

8. Conclusion

A machine-checked proof used to cost six months of a scarce specialist's time, so the world verified almost none of its high-value institutional logic. An autonomous, general-purpose apparatus drops that cost toward the cost of compute, and `nonmathlib4` is the first accumulation of what falls out: kernel-checked theorems in finance, strategy, distributed systems, and cryptography, each self-contained, each axiom-clean, each produced from a natural-language blueprint with no verification engineer in the loop. The theorems compound, a later one citing an earlier one, so the marginal cost of the next verified property keeps falling as the library grows.

A faster prover does not carry the work, and neither does a first verification of business logic, which already exists and in places is commercial. The closed loop does: the Lean compiler as an adversarial ground truth that forces a fallible model to discard its hallucinations, a governance kernel the model cannot influence, certificates a skeptic can re-run. So does the discipline of saying where the guarantee stops. The kernel certifies that the proof follows from the specification; the blueprint, argued in prose, carries the specification's meaning, and the Shamir case shows what happens at that seam. Owning the seam separates a certificate from a claim.

The next demonstration is one of scale. A contested-spec covenant is already filed: a restricted-payments clause from a live SEC EDGAR agreement, its accounting abstracted by a human, its faithfulness argued in a companion certificate. What the apparatus does not yet do is read the agreement itself. Today a human draws the abstraction boundary and hands the engine a scoped blueprint; the thousand-page credit agreement, with the machine deciding for itself which clauses to formalize, is the stage it does not yet exercise at scale, and the edge of what it can do today. A second direction matters as much for adoption: a theorem library is hard to use, so a navigable surface over the filed results, one that lets a researcher see which mechanism cites which and where a constraint would break a baseline, turns the library from an archive into a workbench. Neither is filed yet. Both are the work in front of us.

The name states the scope. Mathlib is the library of mathematics. `nonmathlib4` is the library of everything else that turns out to be provable: the mechanisms and institutional logic that stayed unformalized because the people who could verify it and the people who understood it were rarely the same, and that a machine can now formalize from a written blueprint.

References

- Backes, Bolignano, Cook, Dodge, Gacek, Luckow, Rungta, Tkachuk, Varming. *Semantic-based Automated Reasoning for AWS Access Policies using SMT*. FMCAD 2018. <https://www.amazon.science/publications/semantic-based-automated-reasoning-for-aws-access-policies-using-smt>
- Bei, Ma, Jing, Fu, Tang. *EconCSLib: A Lean Library for Computational Economics and AI-Assisted Research*. AI-EconCS '26 (EC'26 workshop). arXiv:2606.16144. <https://arxiv.org>

[g/abs/2606.16144](https://arxiv.org/abs/2606.16144)

- Coelho. *A Formally Verified Library of Mathematical Finance in Lean 4*. 2026. arXiv:2606.01356. <https://arxiv.org/abs/2606.01356>
- LeanMarathon: *Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*. 2026. arXiv:2606.05400. <https://arxiv.org/abs/2606.05400>
- Buzzard, Commelin, Massot. *Formalising Perfectoid Spaces*. CPP 2020. <https://doi.org/10.1145/3372885.3373830>
- Certora. *The Certora Prover*. <https://github.com/Certora/CertoraProver>
- Cook (AWS). *Proving security at scale with automated reasoning*. 2019. <https://www.allthingsdistributed.com/2019/06/proving-security-at-scale-with-automated-reasoning.html>
- de Moura, Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- Google DeepMind. *AI achieves silver-medal standard solving International Mathematical Olympiad problems* (AlphaProof / AlphaGeometry 2). 2024. <https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/>
- Google DeepMind et al. *Olympiad-level formal mathematical reasoning with reinforcement learning*. Nature, 2025. <https://www.nature.com/articles/s41586-025-09833-y>
- Jiang, Welleck, Zhou, Li, Liu, Jamnik, Lacroix, Wu, Lample. *Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs*. ICLR 2023. <https://arxiv.org/abs/2210.12283>
- Jiang, Li, Zhu, Jamnik, et al. *Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers*. NeurIPS 2022. arXiv:2205.10893. <https://arxiv.org/abs/2205.10893>
- Klein, Elphinstone, Heiser, Andronick, Cock, Derrin, Elkaduwe, Engelhardt, Kolanski, Norrish, Sewell, Tuch, Winwood. *seL4: Formal Verification of an OS Kernel*. SOSP 2009. <https://dl.acm.org/doi/10.1145/1629575.1629596>
- Klein, Andronick, Elphinstone, Murray, Sewell, Kolanski, Heiser. *Comprehensive Formal Verification of an OS Microkernel*. ACM TOCS 2014. <https://doi.org/10.1145/2560537>
- Lample, Lacroix, Lachaux, Rodriguez, Hayat, Lavril, Ebner, Martinet. *HyperTree Proof Search for Neural Theorem Proving*. NeurIPS 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/a8901c5e85fb8e1823bbf0f755053672-Abstract-Conference.html
- mathlib Community. *The Lean Mathematical Library*. CPP 2020. <https://doi.org/10.1145/3372885.3373824>
- Merigoux, Chataing, Protzenko. *Catala: A Programming Language for the Law*. ICFP 2021. arXiv:2103.03198. <https://arxiv.org/abs/2103.03198>
- Mikula, Tworowski, Jiang, et al. *Magnushammer: A Transformer-Based Approach to Premise Selection*. arXiv:2303.04488. <https://arxiv.org/abs/2303.04488>
- Passmore et al. (Aesthetic Integration). *Formal Verification of Financial Algorithms* (Imandra). CADE 2017. https://link.springer.com/chapter/10.1007/978-3-319-63046-5_3
- Hildenbrandt, Saxena, Rodrigues, Zhu, Daian, Guth, Moore, Park, Zhang, Stefanescu, Roşu. *KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine*. IEEE CSF 2018. <https://github.com/runtimeverification/evm-semantics>
- Wu, Jiang, Li, Rabe, Staats, Jamnik, Szegedy. *Autoformalization with Large Language Models*. NeurIPS 2022. arXiv:2205.12615. <https://arxiv.org/abs/2205.12615>
- Yang, Swope, Gu, Chalamala, Song, Yu, Godil, Prenger, Anandkumar. *LeanDojo: Theorem*

Proving with Retrieval-Augmented Language Models. NeurIPS 2023. arXiv:2306.15626. <https://arxiv.org/abs/2306.15626>